

HillGo

Platform Architecture Compendium

Four Mobile Applications · Admin Operations Panel · Public Website
Laravel Backend · Database Architecture · Security Model · System Integration

Confidential - Client Architecture Deliverable
Version 1.0 · August 2026

Contents

1. Executive Overview
2. Architecture 1 - Customer Application
3. Architecture 2 - Rider Application
4. Architecture 3 - Merchant Application
5. Architecture 4 - Courier Application
6. Architecture 5 - Admin Panel
7. Architecture 6 - Public Website
8. Architecture 7 - Database
9. Backend Architecture
10. End-to-End System Integration
11. Security Architecture (Layered)
12. Appendix - Feature Coverage Matrix

1. Executive Overview

HillGo is a Bangladesh multi-service logistics and lifestyle platform spanning ride-hailing, food delivery, parcel courier, merchant commerce, hotels, and rentals. The platform is delivered as seven primary architecture surfaces: four Flutter applications (Customer, Rider, Merchant, Courier), an Admin operations panel, a Public marketing website, and a unified relational database - all coordinated through a single Laravel API backend.

All monetary and operational framing uses Bangladeshi Taka (৳). Clients authenticate with role-scoped Sanctum tokens. Shared domain services enforce Region Lock, pricing, wallet ledger settlement, OTP custody, dispatch, notifications, and auditability.

HillGo — Platform System Architecture

Four mobile apps - Admin panel - Public website - Laravel API - MySQL

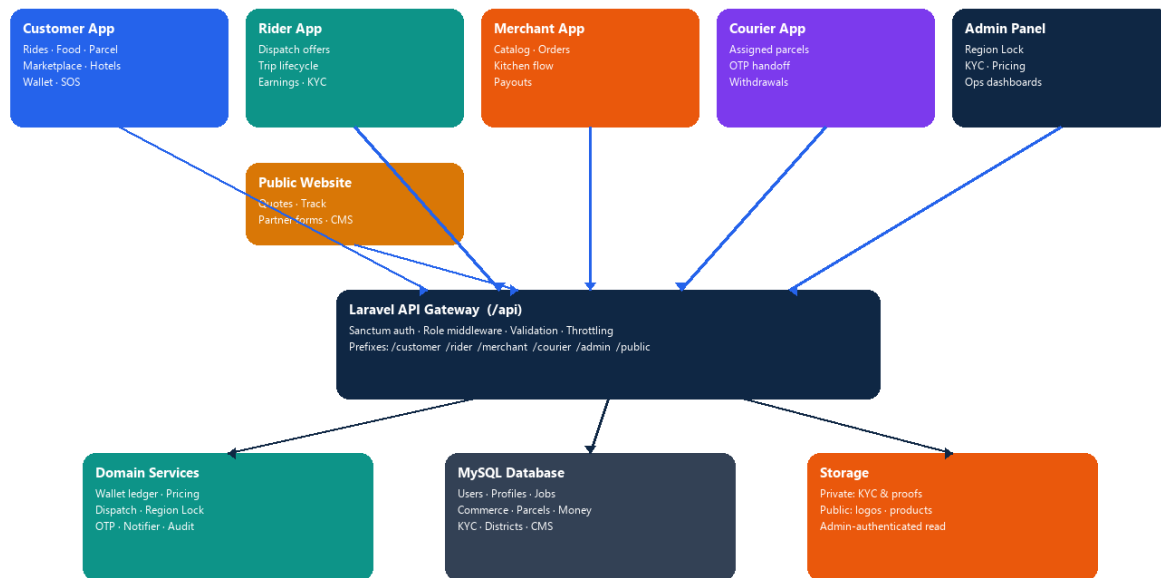


Figure 1 — System of systems overview

Figure 1. Platform system-of-systems overview.

1.1 Architecture inventory

#	Architecture surface	Technology	Primary consumers
1	Customer Application	Flutter	End customers
2	Rider Application	Flutter	Driver / delivery partners
3	Merchant Application	Flutter	Store operators
4	Courier Application	Flutter	Parcel agents
5	Admin Panel	HTML/JS SPA	HillGo operations
6	Public Website	Static web	Prospects & partners
7	Database	MySQL / InnoDB	All services via API

2. Architecture 1 - Customer Application

The Customer application is the consumer super-app. It presents a unified shell for mobility, food, parcels, marketplace, hotels, rentals, wallet, loyalty, and safety (SOS).

2.1 Structural layers

- Presentation: feature screens, main shell navigation, theming, shared widgets.
- Application: route map and feature orchestration for booking and tracking journeys.
- Integration: ApiClient with secure Sanctum token storage; modular API services per domain.
- Network: HTTPS calls exclusively to `/api/customer/*` (plus shared public helpers where needed).

Customer App — Layered Architecture

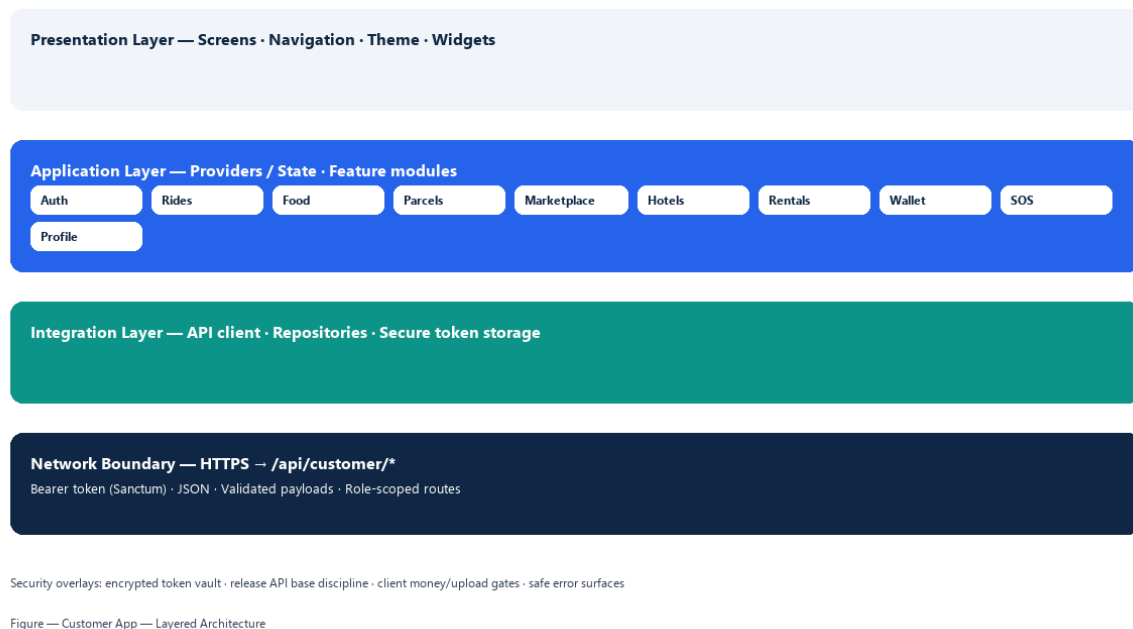


Figure 2. Customer application layered architecture.

2.2 Capability map

- Identity: registration, login, OTP verification, profile, addresses, payment methods.
- Mobility: fare quote, ride booking, live tracking, rating.
- Food & commerce: restaurant browsing, cart checkout, marketplace products, order history.
- Parcels: quote, book, track.
- Lifestyle: hotels and rental vehicle bookings.
- Money & loyalty: wallet top-up/history, rewards, promos.
- Safety: SOS contacts and alerts; notifications inbox.

2.3 Data flow

The customer authenticates under `/api/customer/auth/*`, persists the bearer token in the device secure vault, then executes all domain actions through role-scoped customer endpoints. Server-side pricing and Region Lock decide eligibility and amounts; the app never acts as the source of truth for settlement.

3. Architecture 2 - Rider Application

The Rider application enables driver partners to complete KYC, go online, receive dispatch offers (ride / food / parcel jobs), navigate active trips, and cash out earnings.

Rider App — Layered Architecture

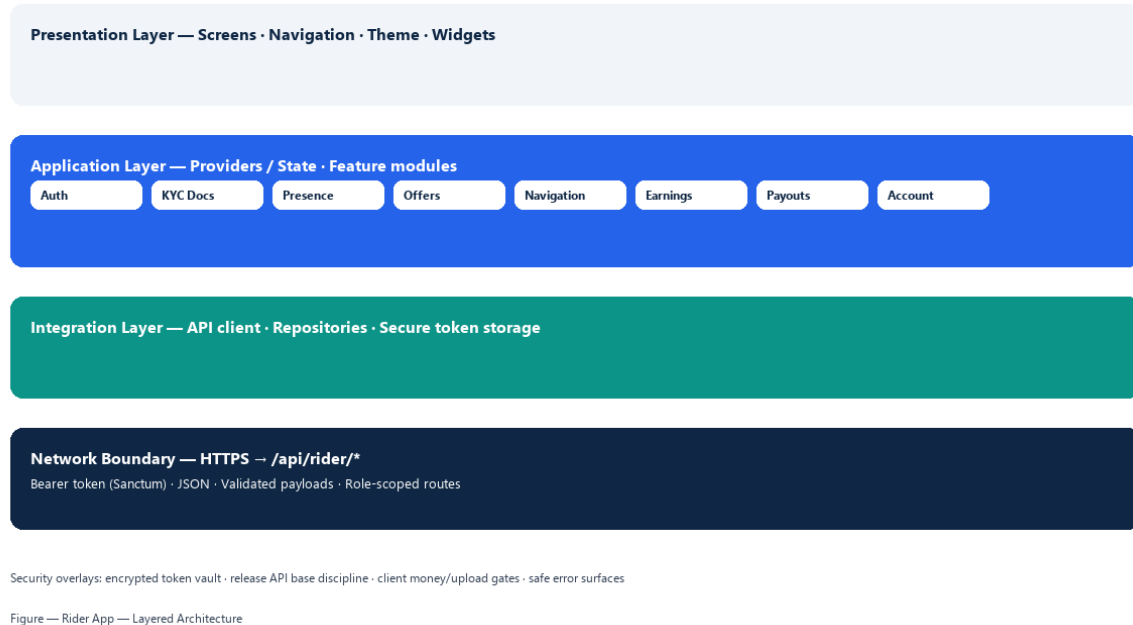


Figure 3. Rider application layered architecture.

3.1 Capability map

- Onboarding & KYC: personal info, vehicle info, document upload, verification status.
- Presence: online/offline toggle and location updates for dispatch.
- Trip lifecycle: incoming offer, accept/decline, navigate, complete.
- Earnings: balance, history, payout / cash-out requests.
- Account: profile and operational preferences.

3.2 Data flow

After /api/rider/auth/* login and KYC submission, the rider posts presence to the backend. Dispatch creates trip offers; the rider advances trip state via /api/rider/trips/*. Routing guidance is obtained through the HillGo API route proxy so live coordinates remain within the HillGo trust boundary.

4. Architecture 3 - Merchant Application

The Merchant (Vendor) application covers store onboarding, catalog management, kitchen order flow, branding assets, reviews, and payout operations.

Merchant App — Layered Architecture

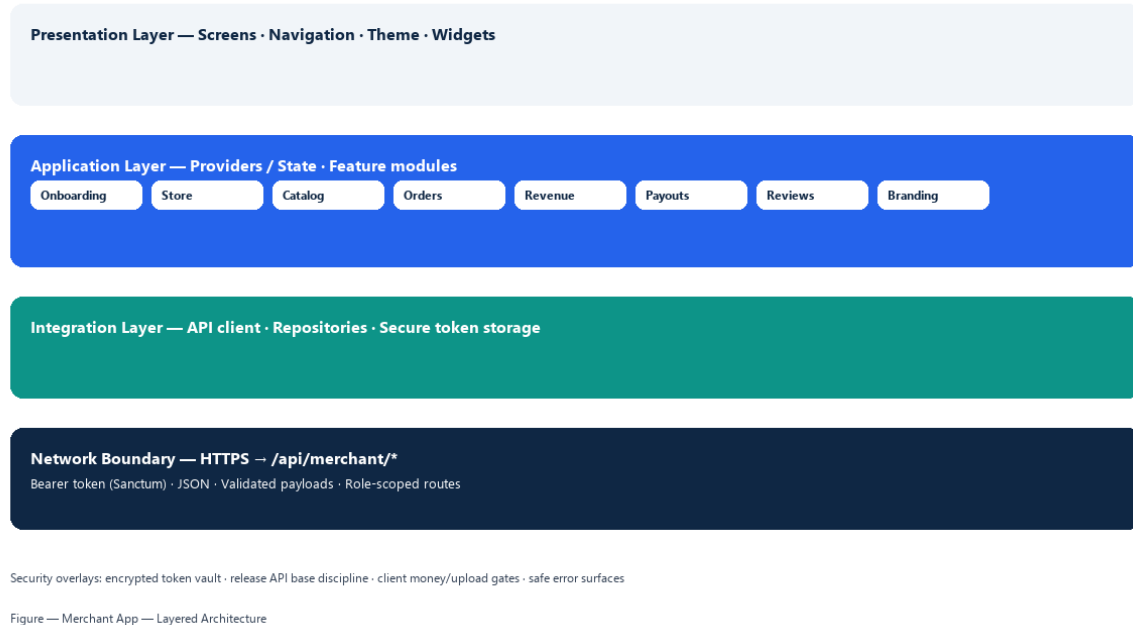


Figure 4. Merchant application layered architecture.

4.1 Capability map

- Onboarding: business profile, trade license / NID, storefront and logo media.
- Store hub: open/accepting status, hours, branding updates.
- Catalog: categories and products with image upload.
- Orders: accept → prepare → ready → delivered / reject.
- Revenue: summaries, transactions, early payout requests.
- Reviews: read and reply; settings synchronized to the server.

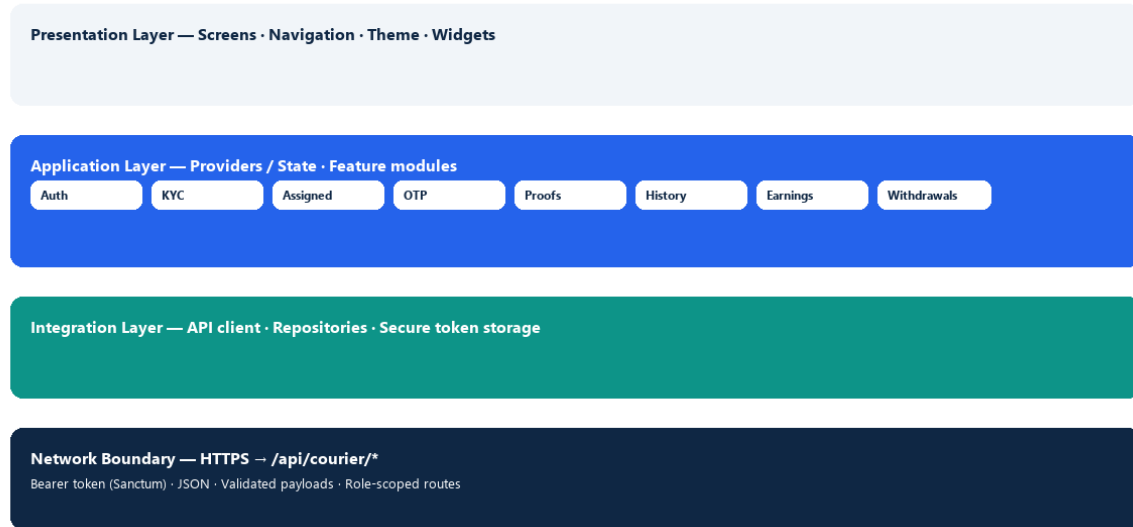
4.2 Data flow

Merchants authenticate at /api/merchant/auth/*, submit onboarding for admin review, then manage store and catalog through /api/merchant/*. Product and branding images land on the public disk; KYC documents remain on the private disk for admin review only.

5. Architecture 4 - Courier Application

The Courier application equips parcel agents with assignment queues, OTP-secured pickup/delivery, optional proof capture, earnings, incentives, and withdrawals.

Courier App — Layered Architecture



Security overlays: encrypted token vault · release API base discipline · client money/upload gates · safe error surfaces

Figure — Courier App — Layered Architecture

Figure 5. Courier application layered architecture.

5.1 Capability map

- Registration & KYC document upload (license, NID, registration).
- Assigned parcel queue and parcel detail.
- Pickup / delivery OTP confirmation with attempt controls.
- Delivery proof photo/signature when required.
- History, earnings dashboard, incentives, withdrawal requests.
- Vehicle, bank, and notification preferences.

5.2 Data flow

Agents authenticate under `/api/courier/auth/*`, receive assignments from `/api/courier/parcels/*`, confirm custody with OTP, and may upload private proof files. Earnings and withdrawals settle through courier earnings endpoints under admin oversight.

6. Architecture 5 - Admin Panel

The Admin Panel is a hash-routed single-page application used by HillGo operations. It provides Region Lock control, KYC queues, live operational dashboards, pricing governance, payouts, and public-web inbox/CMS tools.

Admin Panel — SPA Architecture

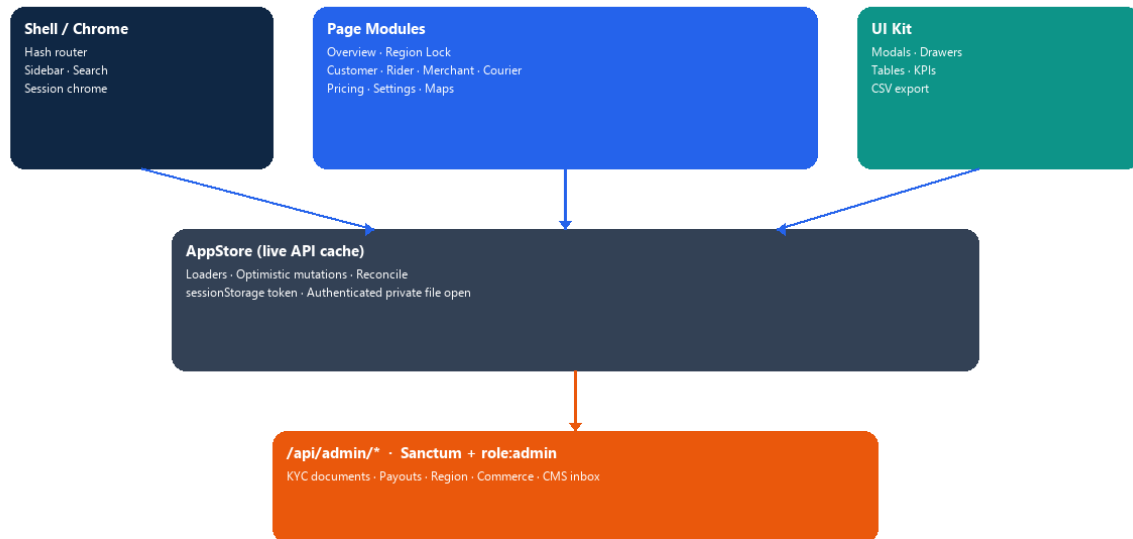


Figure — Admin Panel architecture

Figure 6. Admin Panel SPA architecture.

6.1 Structural pattern

- Shell: sidebar navigation, global search, session chrome.
- Router: client-side hash routes for overview, region, customer, rider, merchant, courier, settings.
- AppStore: in-memory cache with optimistic updates reconciled against Laravel responses.
- UI toolkit: tables, KPIs, drawers, modals, map overlays, CSV export.
- Private media: authenticated open of KYC and proof files (Bearer-protected).

6.2 Operational domains

- Region Lock - divisions/districts open/closed and per-role allow flags.
- Customer ops - directory, rides, food orders, parcels, SOS, wallet adjustments.
- Rider ops - KYC, trips, live map, salary/payouts, rider pricing.
- Merchant ops - onboarding review, stores, orders, payouts, merchant pricing.
- Courier ops - agent KYC, parcels, withdrawals, incentives, courier pricing.
- Platform - settings, activity log, overview KPIs, public web content inbox.

7. Architecture 6 - Public Website

The Public Website is the marketing and discovery surface for HillGo services and partner acquisition. It is a static front end calling unauthenticated, rate-limited public API endpoints.

Public Website — Architecture

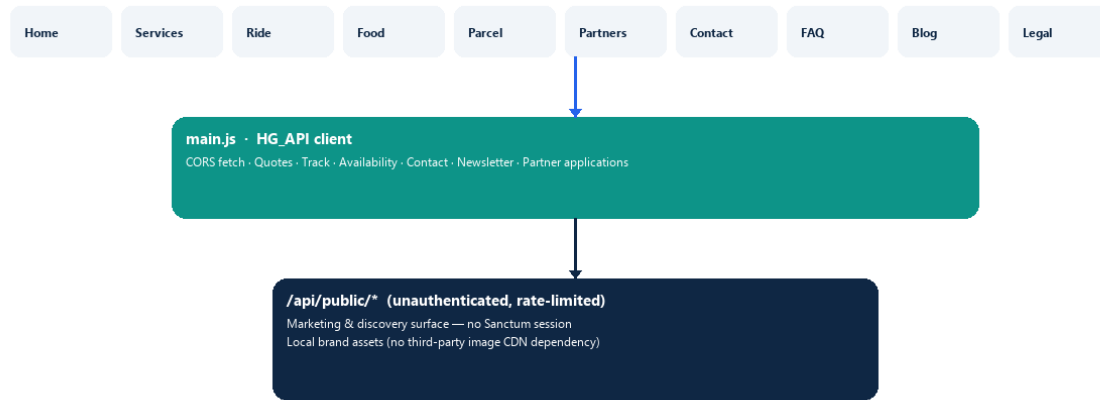


Figure — Public Website architecture

Figure 7. Public Website architecture.

7.1 Surfaces

- Service storytelling: home, about, services, ride, food, parcel.
- Partner acquisition: merchant and driver registration funnels.
- Trust & content: FAQ, blog, terms, privacy.
- Interactive tools: fare quote calculator, parcel tracking, city availability, contact, newsletter.

7.2 Data flow

Browsers invoke `/api/public/*` with CORS-aware JSON fetch. No Sanctum session is established. Content aggregates (testimonials, contact strip, pricing display) and operational utilities (quotes, track, availability) are served by the public controller layer.

8. Architecture 7 - Database

HillGo persists operational state in a MySQL (InnoDB) relational schema. The schema is organized by domain rather than by client, enabling one source of truth for identity, jobs, commerce, money, KYC, geography, lifestyle bookings, safety, and CMS.

Database Architecture — Domain Map



Figure — Logical database domains (MySQL / InnoDB)

Figure 8. Database domain map.

8.1 Core domains

Domain	Representative tables	Purpose
Identity	users, personal_access_tokens, otp_codes	Accounts, sessions, OTP
Profiles	customer_profiles, rider_profiles, courier_profiles, stores, addresses	Role-specific master data
Mobility	rides, trips	Customer rides and partner job offers
Commerce	orders, order_items, products, product_categories, reviews	Food & marketplace
Parcels	parcels, parcel_otp_logs, parcel_proofs	Courier logistics
Money	wallet_transactions, *_payouts, courier_withdrawals, pricing_*	Ledger & tariffs
KYC	rider_documents, courier_documents, merchant_onboardings	Partner verification
Region	divisions, districts	Geographic enablement (Region Lock)
Lifestyle	hotels, hotel_bookings, rental_*, loyalty_*, promos	Ancillary services
Safety/CMS	sos_*, app_notifications, faqs, blog_posts, inquiries	Safety & public content

8.2 Integrity principles

- Role is an attribute of the user record and is enforced server-side on every request.
- Wallet mutations occur through a single ledger path (balance + wallet_transactions + audit).
- District flags (status + allow_customer/rider/merchant/courier) gate product availability.
- Soft-delete and audit trails support operational forensics without losing history.
- Hot-path indexes support list/search workloads for trips, parcels, orders, and KYC queues.

9. Backend Architecture

The backend is a Laravel API that exposes a single /api surface partitioned by role. Controllers are thin HTTP adapters; domain rules live in dedicated services (Wallet, RegionLock, OtpService, PricingService, Dispatch, Notifier, Audit).

Backend Architecture — Laravel API



Figure — Backend controller & service layout

Figure 9. Backend controller and service layout.

9.1 Cross-cutting mechanics

- Authentication: Laravel Sanctum personal access tokens.
- Authorization: role middleware binds the authenticated user to admin/customer/rider/merchant/courier_agent.
- Validation: request payloads validated before persistence.
- Queues: OTP delivery and app notifications executed via queued jobs.
- Filesystems: private disk for KYC/proofs; public disk for storefront/product media via storage link.
- Observability: activity log and pricing audit records for governance.

9.2 Storage architecture

Sensitive identity documents and delivery proofs are written to the private local disk and are never exposed by a public URL. Admin download routes stream files only after Sanctum + admin role checks. Merchant logos, banners, and product images are stored on the public disk and served through /storage/... after the storage link is established.

10. End-to-End System Integration

Although each client has its own architecture, runtime value is created when they collaborate through the API and database. A customer intent becomes a priced, region-checked record; dispatch allocates a partner; partners advance job state; wallet settlement and notifications close the loop; admin governs KYC, pricing, and payouts.

End-to-End Runtime — Example Cross-Role Flow

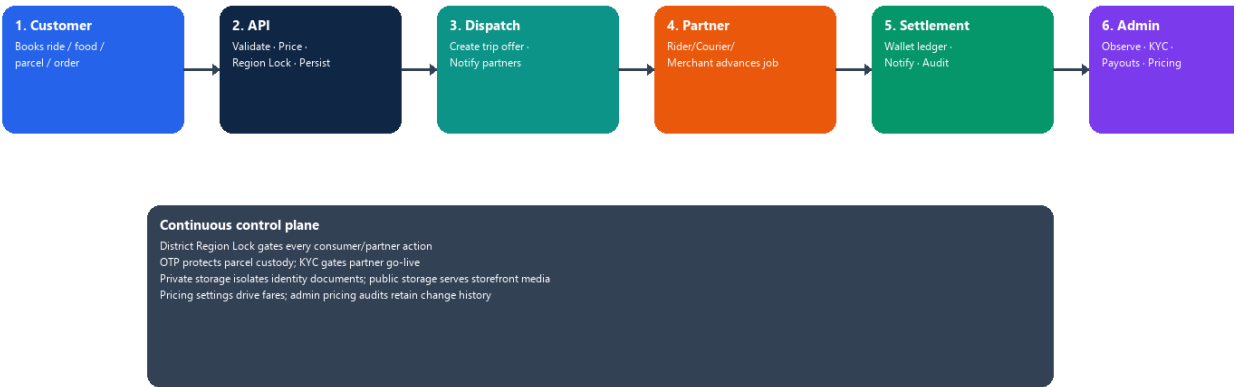


Figure — Cross-application runtime flow

Figure 10. Cross-application runtime flow.

10.1 API prefix map

Prefix	Client	Access gate
/api/public/*	Public Website (+ shared utilities)	Throttle only
/api/customer/*	Customer App	Sanctum + role:customer
/api/rider/*	Rider App	Sanctum + role:rider
/api/merchant/*	Merchant App	Sanctum + role:merchant
/api/courier/*	Courier App	Sanctum + role:courier_agent
/api/admin/*	Admin Panel	Sanctum + role:admin

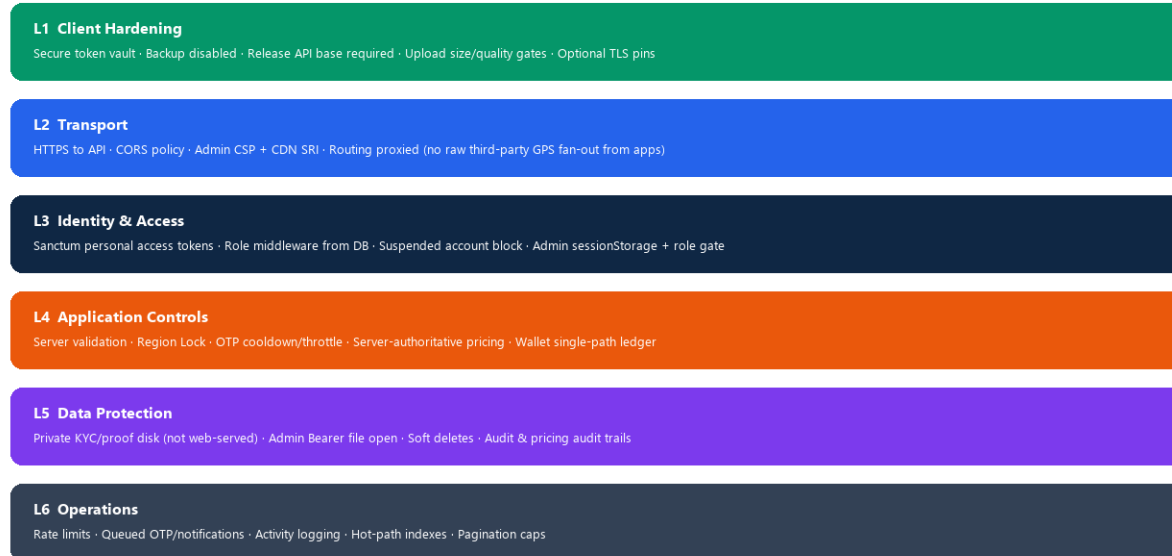
10.2 Shared control plane

- Region Lock decides where each role may operate.
- Pricing settings decide fares and fees displayed and charged.
- Dispatch links customer demand to rider/courier supply.
- Wallet ledger is the only path that mutates balances.
- OTP protects parcel custody transitions.
- KYC status gates partner go-live across rider, merchant, and courier.

11. Security Architecture (Layered)

Security is expressed as concentric operational layers - from the device edge to identity, application controls, sensitive data isolation, and operations. The following diagram is the security architecture drawing for this platform.

Security Architecture — Defense Layers



Outer rings protect devices & transport; inner rings protect identity, money, KYC, and operational integrity.

Figure — Layered security model

Figure 11. Layered security model.

11.1 Layer summaries

- L1 Client hardening - secure token storage, backup restrictions, release configuration discipline, upload limits.
- L2 Transport - HTTPS, CORS, admin content security policy / SRI, proxied routing.
- L3 Identity & access - Sanctum tokens, DB-backed roles, suspension enforcement, admin session controls.
- L4 Application controls - validation, Region Lock, OTP throttles, server pricing, wallet ledger.
- L5 Data protection - private KYC/proof disk, authenticated admin file access, soft deletes, audits.
- L6 Operations - rate limits, queued notifications, activity logs, pagination caps, indexing.

12. Appendix - Feature Coverage Matrix

Capability	Customer	Rider	Merchant	Courier	Admin	Public
Authentication / OTP	Yes	Yes	Yes	Yes	Yes	-
Region Lock awareness	Yes	Yes	Yes	Yes	Controls	Availability
Ride / trip jobs	Book/track	Execute	-	-	Oversee	Quote
Food / marketplace	Order	Deliver jobs	Catalog/kitchen	-	Oversee	Story
Parcels	Book/track	Optional jobs	-	Execute + OTP	Oversee	Track
Wallet / payouts	Wallet	Cash-out	Payouts	Withdraw	Approve/adjust	-
KYC documents	-	Upload	Onboarding docs	Upload	Verify + view	-
CMS / marketing	-	-	-	-	Inbox/CMS	Publish

Document close

This compendium defines the seven architecture surfaces of HillGo together with the backend integration fabric and the layered security model. It is intended as the reference blueprint for stakeholders reviewing how the mobile applications, admin panel, public website, database, and API operate as one platform.